Northern University of Business and Technology Khulna
Department of Computer Science and Engineering

AI INTEGRATION & INTELLIGENT FEATURES REPORT

Project Title: AI-Powered Resume Analyzer and Career Copilot

Course Title: Mobile Computing Lab

Course Code: CSE4204

Submitted to:
Md. Riaz Mahmud
Assistant Professor
Department of Computer Science and Engineering
Northern University of Business and Technology, Khulna

TEAM INFORMATION SHEET

Field	Details
Official Team Name	CSE4204-8A-T05
Section	8A
Project Title	AI-Powered Resume Analyzer and Career Copilot
Team Member List with IDs	1. Mahammad Hasan (ID:11220320828) -Team Leader, Frontend Architecture Lead 2. Shanawaz Sakib (ID:11220320916) - Core System Integration Engineer 3. Rasel Ratul (ID:11220320827) - UI-UX & Tailwind Matrix Designer 4. Ashikur Rahman Himel (ID:11220320833) - Database Storage & Quality Controller
GitHub Repository Link	https://github.com/mahamud894/AI_Resume_Analyzer
Figma Link	https://www.figma.com/design/Ls0B3DIjxRKrP2RIzYQxPG/AI-Resume-Analyzer?node-id=2-2&t=lFEVQCoUHt0JSumI-1

1. Executive Summary & Problem-Solving Capability

1.1 Why AI is Required in This Domain

In the modern recruitment landscape, job seekers face a massive "black box" obstacle: Applicant Tracking Systems (ATS) filter out over 75% of candidate resumes before a human recruiter ever reads them. Most applicants suffer from repeated automated rejections without understanding whether their application failed due to missing technical keywords, poor structural formatting, unoptimized tone, or skill gap mismatches relative to the target job scope. Manual resume editing and hiring consultation services are slow, highly expensive, and inaccessible to the vast majority of fresh graduates.

1.2 Core Real-World Problem Solved

Our application, **RESUMIND**, implements Large Language Models (LLMs) to perform automated, multi-dimensional candidate-to-job matching. Instead of naive keyword counting, our AI engine semantically analyzes multi-page PDF candidate resumes against explicit target job descriptions. It generates deterministic ATS compatibility scores out of 100, categorizes critical skill shortages, and delivers actionable, step-by-step career improvement tips—all processed within seconds.

2. Selected AI Service & Model Architecture

2.1 AI Platform & Foundation Models

- **Selected Infrastructure:** Serverless Client-to-Cloud AI Gateway via **Puter.js SDK**.
- **Underlying Foundation Models:** Interfaced with **GPT-4o/ Claude 3.5 Sonnet / llama-3.3-70b-versatile** runtime orchestration nodes.
- **Selection Rationale:** Operates on a privacy-first, serverless architecture where browser SDK calls interface directly with cloud AI proxies. This design eliminates monolithic backend server costs, prevents master API key exposure in public repositories, and provides zero-maintenance, high-throughput LLM evaluation pipelines.
- **Environment Security:** Production API credentials and routing proxies are strictly handled at the Puter cloud boundary. No raw environment variables or master keys are

exposed within client repositories.

3. End-to-End AI Workflow Design

The end-to-end data pipeline processing candidate PDFs into real-time analytical feedback follows a strict 7-layer architecture:

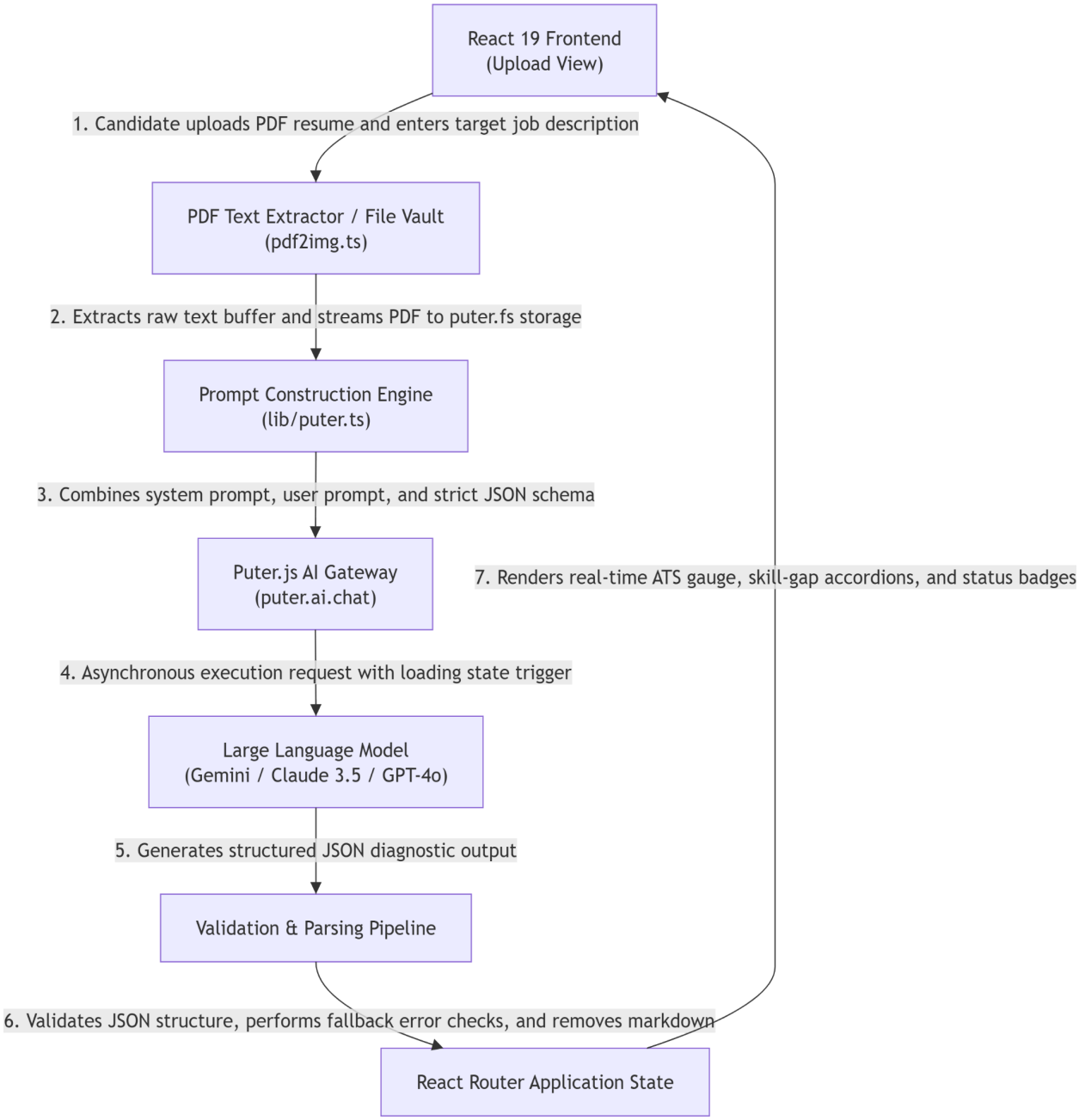

Figure: AI Workflow Diagram

Detailed Workflow Step Breakdown:

1. **User Data Input:** The candidate uploads a multi-page PDF resume (up to 10MB) via react-dropzone and inputs target Company Name, Job Title, and raw Job Description text.
2. **Document Extraction & Storage:** The client extracts raw text strings using pdfjs-dist (lib/pdf2img.ts) while streaming the binary document buffer securely into Puter's cloud filesystem (puter.fs.write).
3. **Prompt Injection:** The system compiles a strict system prompt and user context, embedding strict JSON output formatting rules.
4. **Cloud Execution:** The payload is passed directly to puter.ai.chat() via an asynchronous controller with loading indicator triggers.
5. **LLM Evaluation:** The AI model evaluates semantic match, ATS compliance, skill alignment, formatting, and tone.
6. **Response Cleaning & Sanitization:** The frontend catches raw response strings, strips markdown wrappers (e.g. json ...), and parses the data into type-safe TypeScript objects matching types/index.d.ts.
7. **UI State Render:** Results update application states (routes/resume.tsx), triggering dynamic score gauges, alert banners, and collapsible breakdown accordions.

4. Prompt Engineering Strategy

To ensure deterministic, type-safe outputs from the AI core matching our types/index.d.ts schema, we implemented a structured **System Prompt + User Prompt** strategy enforcing strict JSON schema responses.

4.1 System Prompt Design

Example:

You are an expert HR Specialist, Executive Recruiter, and Applicant Tracking System (ATS) Auditor. Your goal is to analyze a candidate's resume against a provided job description and deliver an objective, highly detailed critique.

STRICT RULE: You MUST return ONLY a valid, raw JSON object matching the exact structure below. Do NOT wrap your output in ```json markdown code blocks, do NOT include greetings, intro text, or extra prose.

Required JSON Output Format:

```
{
 "overallScore": number (0-100),
 "ATS": {
  "score": number (0-100),
  "tips": [
   { "type": "good" | "improve", "tip": string }
  ]
 },
 "toneAndStyle": {
  "score": number (0-100),
  "tips": [
   { "type": "good" | "improve", "tip": string, "explanation": string }
  ]
 },
 "content": {
  "score": number (0-100),
  "tips": [
   { "type": "good" | "improve", "tip": string, "explanation": string }
  ]
 },
 "structure": {
  "score": number (0-100),
  "tips": [
   { "type": "good" | "improve", "tip": string, "explanation": string }
  ]
 },
 "skills": {
  "score": number (0-100),
  "tips": [
   { "type": "good" | "improve", "tip": string, "explanation": string }
  ]
```

```
  }
}
```

4.2 User Prompt Example

prompt:

Target Company: Appstick Developer Studio

Target Job Title: Full-Stack Developer (Laravel + React)

Job Description:

- Looking for a skilled developer with experience in Laravel 11, React 19, RESTful APIs, Tailwind CSS, and MySQL.
- Must demonstrate clean code practices, Git version control, and asynchronous state management.

Candidate Resume Text:

"Mahammad Hasan - Computer Science Student & Frontend Architecture Lead. Experienced in React.js, TypeScript, Tailwind CSS, Zustand, and Git. Skilled in integrating Puter.js serverless SDKs, REST APIs, and building responsive web interfaces."

Evaluate this candidate against the job description and return the JSON payload.

4.3 Expected AI JSON Output

```
{
  "overallScore": 82,
  "ATS": {
    "score": 85,
    "tips": [
      { "type": "good", "tip": "Strong frontend technology stack matches job requirements directly." },
      { "type": "improve", "tip": "Missing explicit backend keywords like Laravel and MySQL." }
    ]
  },
  "toneAndStyle": {
```

```
  "score": 90,
  "tips": [
    { "type": "good", "tip": "Professional tone with clear technical leadership titles.", "explanation": "Demonstrates team ownership and clarity." }
  ]
 },
 "content": {
  "score": 78,
  "tips": [
    { "type": "improve", "tip": "Quantify project impact with metrics.", "explanation": "Add metrics like % performance boost or user scaling." }
  ]
 },
 "structure": {
  "score": 88,
  "tips": [
    { "type": "good", "tip": "Clean, highly scannable layout flow.", "explanation": "Easily parsed by automated ATS parsers." }
  ]
 },
 "skills": {
  "score": 75,
  "tips": [
    { "type": "improve", "tip": "Include relational database and PHP/Laravel concepts to maximize match score.", "explanation": "Target role is full-stack." }
  ]
 }
}
```


5. AI Response Handling & Output Validation

To prevent application crashes or distorted UI layouts caused by unexpected AI model outputs, we engineered robust client-side validation routines in lib/puter.ts:

- **Success Responses:** Parsed into React Router / Local state variables, instantly updating score meters and diagnostic lists.
- **Markdown Sanitization:** Automatically removes raw markdown fences (json ...) using regex matching prior to JSON.parse() execution.
- **Malformed / Invalid JSON Catch:** Wrapped in try-catch blocks. If JSON parsing fails, a local fallback parser constructs a default structured error object without breaking page rendering.
- **Empty / Null Guarding:** Verifies that returned prompt fields are populated. If text strings are missing, default placeholders are injected dynamically.
- **Timeout & Network Failures:** If puter.ai.chat() fails or exceeds timeout limits, an alert banner displays a user-friendly error message: *"AI evaluation service is temporarily busy. Please retry in a few seconds."*
- **Rate Limits:** Implements execution key disabled states (disabled={isLoading}) on submission buttons to prevent accidental rapid re-triggers.

6. Current Limitations & Future Improvements

6.1 Current Limitations

- **Scanned PDF Text Loss:** PDFs created purely as flattened images (scanned documents) require an additional OCR pre-processing step before text can be extracted.
- **Token Limits on Long Resumes:** Extremely lengthy multi-page resumes (>15 pages) may exceed prompt context limits.

6.2 Future Improvements & Migration Roadmap

- **Express & MongoDB Deployment:** Finalize the Node.js / Express backend with MongoDB Atlas to store candidate historical reports permanently.
- **Multi-LLM Benchmarking:** Allow users to toggle between GPT-4o, Claude 3.5, and Gemini 1.5 Pro to compare different ATS feedback perspectives.
- **1-Click AI Resume Rewriter:** Automatically rewrite weak bullet points using AI based on generated suggestions.

7. Current Development Progress & System Migration Roadmap

Our engineering team has achieved significant development milestones across both the production-ready client interface and the next-generation backend architecture. The current system status and phase-wise migration progress are detailed below:

7.1 Completed Core Features (Production Ready)

- **Client-Side Application Core:** Fully built on **React 19**, **React Router v7**, and **Tailwind CSS v4** with dynamic responsive layouts, dark-mode gradients, and interactive components (ATS.tsx, Details.tsx, ScoreGauge.tsx).
- **Document Processing Pipeline:** Implemented client-side text extraction and PDF canvas rendering using pdfjs-dist (lib/pdf2img.ts).
- **Active AI Orchestration:** Seamlessly integrated puter.ai.chat() in lib/puter.ts to perform multi-dimensional resume critiques using high-context LLMs.
- **Data Sanitization & Error Handling:** Implemented regex-based JSON cleaning (stripping markdown wrappers), fallback error parsers, and loading states to ensure UI stability.
- **Version Control & GitHub Matrix:** Maintained clean commit histories, well-structured directories (frontend/, backend/, database/, documentation/), and comprehensive repository documentation.

7.2 Active Backend & Database Migration (In-Progress)

To transition from a serverless SDK model to an enterprise-ready, fully self-hosted ecosystem, the team is actively executing a planned architectural shift:

- **Custom REST API Server:** Developed a **Node.js + Express.js** backend architecture to handle dedicated endpoints for user authentication, file uploads, and AI prompt dispatches.
- **Database Schema Design:** Configured a **MongoDB Atlas** cluster using **Mongoose ODM**. Schemas for persistent user profiles (User.js) and historical ATS evaluation

reports (Analysis.js) are established and undergoing local integration testing.

- **Server-Side Upload & Parser Streams:** Implemented **Multer middleware** on the Express server to stream binary PDF uploads and extract text buffers server-side, eliminating heavy browser-side dependencies.
- **Environment Credential Isolation:** Encapsulated OpenAI and Google Gemini API keys inside server-side environment variables (process.env.AI_API_KEY), ensuring complete protection against network interception.

7.3 Upcoming Timeline (Target Completion: 1–2 Weeks)

The migration to a custom hosted backend and database is actively underway and on track to be fully completed within the next **1 to 2 weeks**:

1. **Week 1 (API & Database Synchronization):** Complete end-to-end testing of Express REST endpoints with the React 19 frontend, replacing browser-side Puter calls with Axios HTTP payloads.
2. **Week 2 (Final Database Persistence & Live Deployment):** Finalize MongoDB state persistence for candidate analysis history, execute full end-to-end validation across error boundary scenarios, and deploy the unified full-stack application.

8. Required Progress Verification Screenshots

8.1 AI Feature Interface View

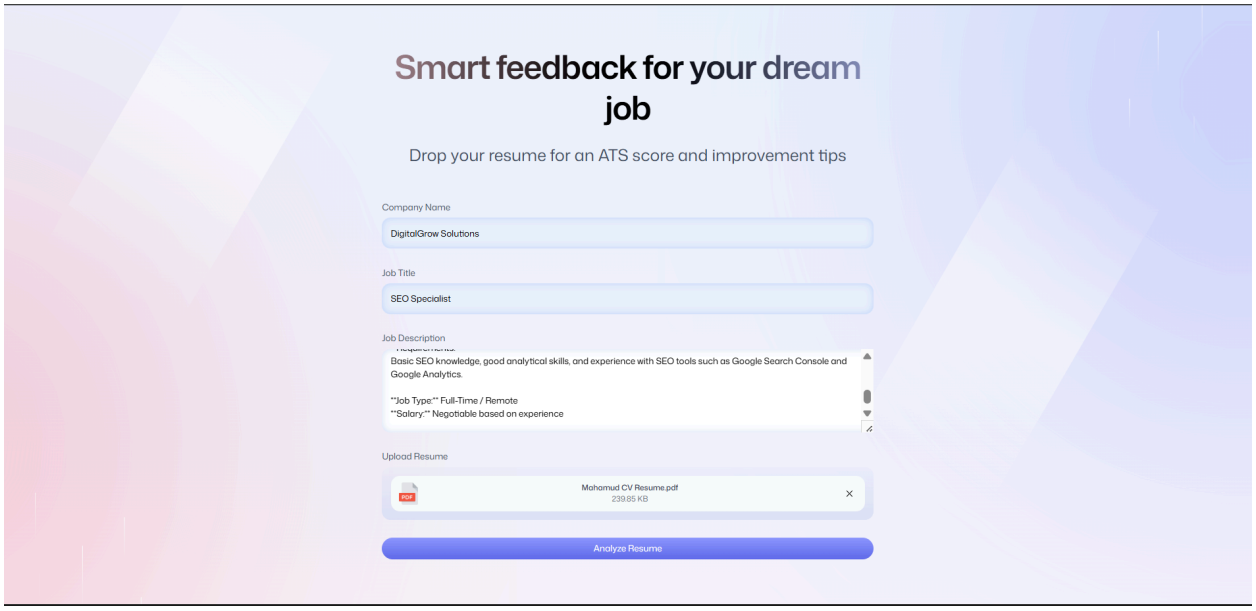

Figure: AI Feature Interface

8.2 AI Interaction & Processing View

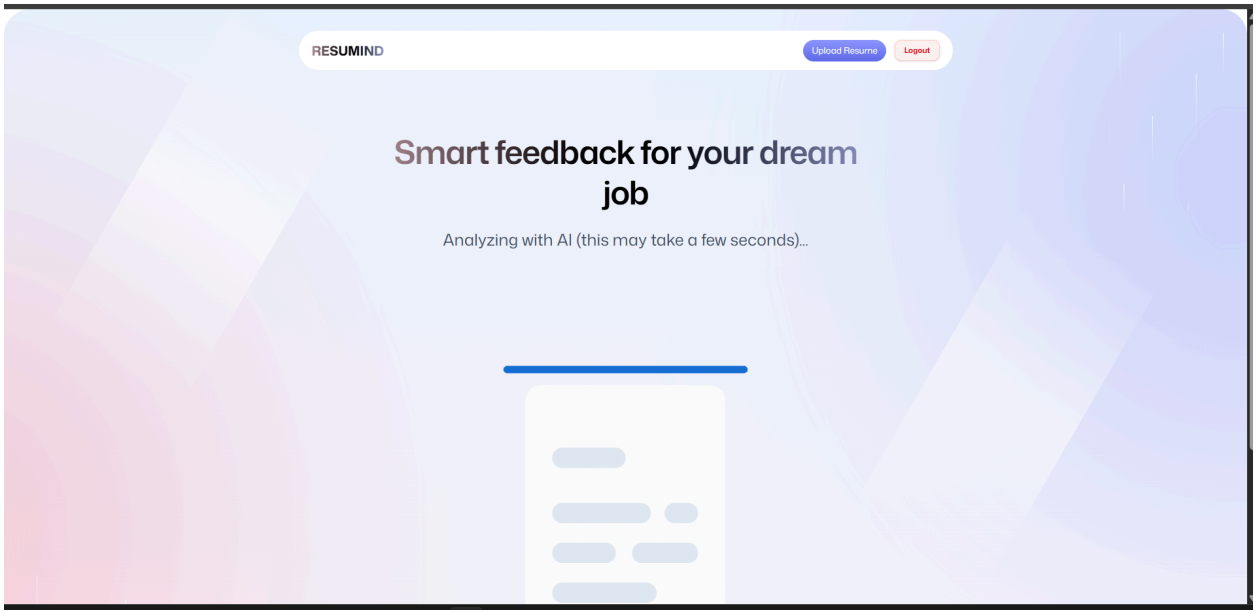

Figure: AI Interaction & Processing View

8.3 AI Dynamic Evaluation Response View

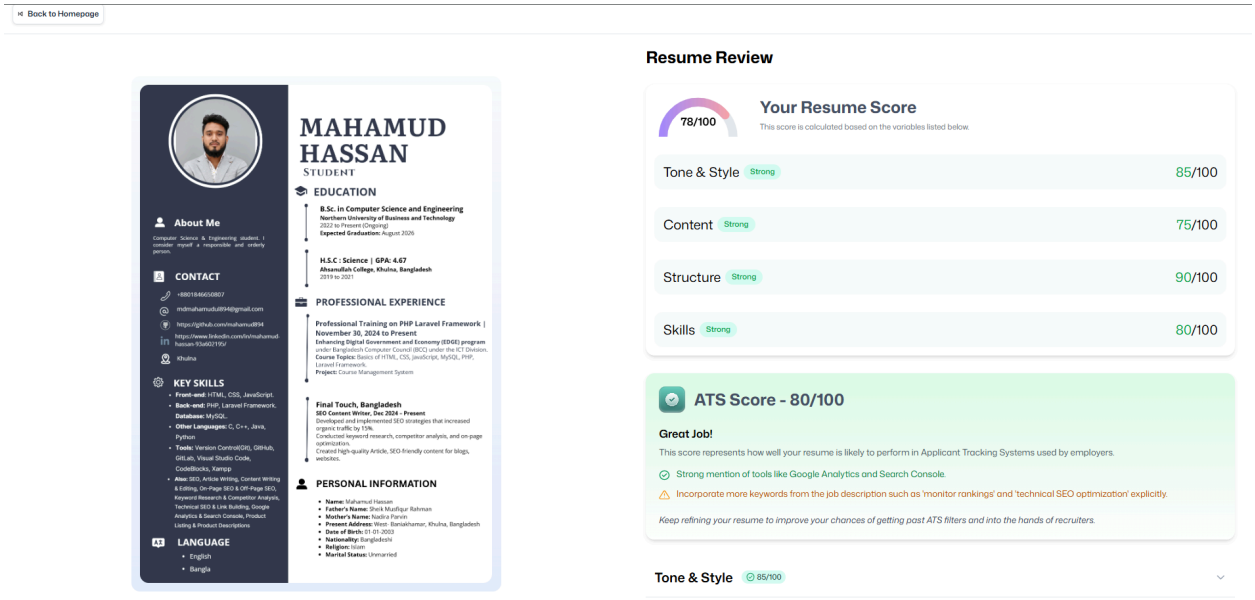

Figure: AI Dynamic Evaluation Response View

8.4 Prompt Engineering & Code Integration View

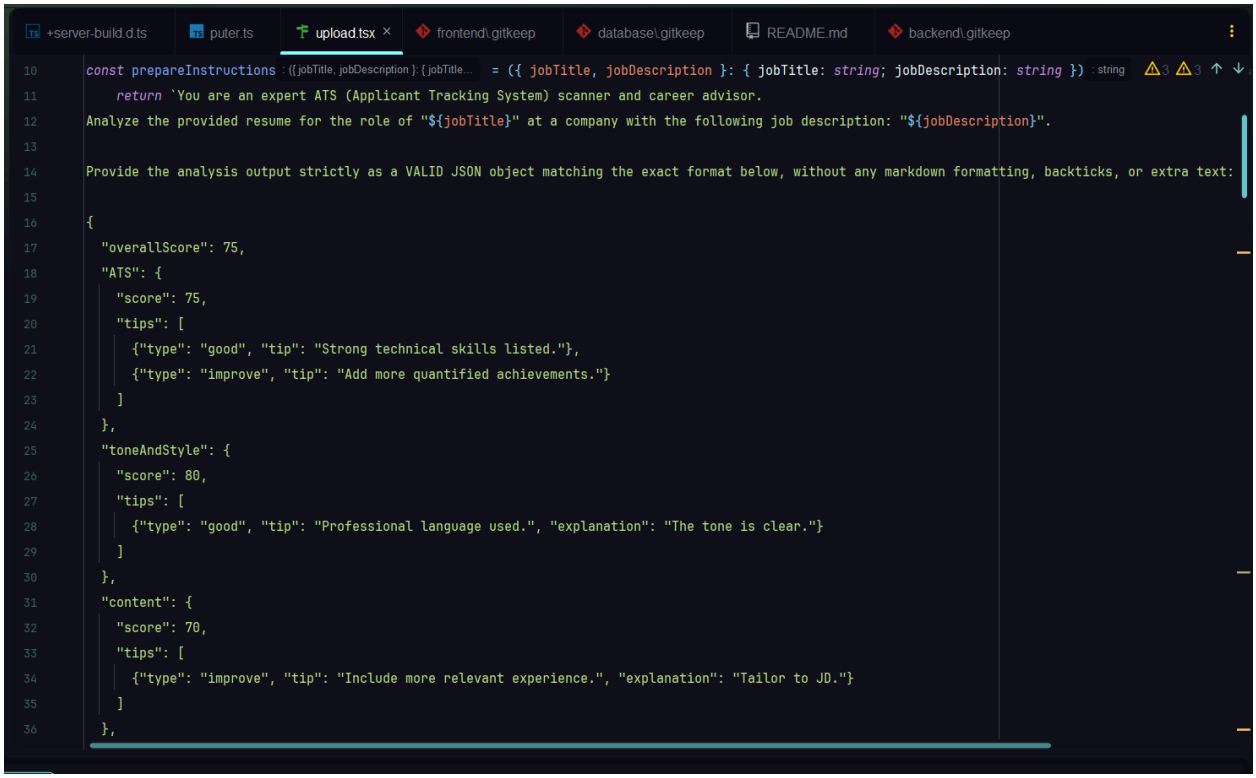

Figure: Prompt Engineering for Ai resume analyzer

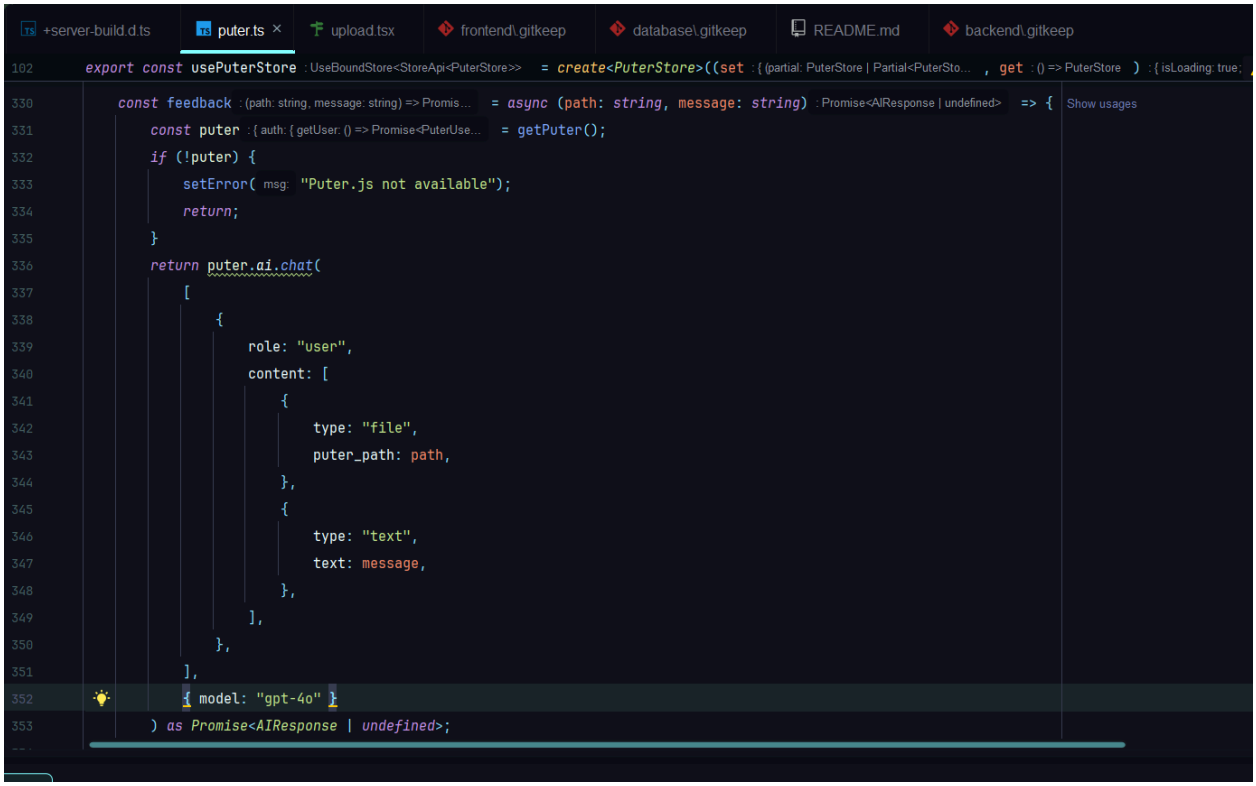

Figure: Ai model calling and Code Integration View for Ai resume analyzer

8.5 Error Handling & Validation State View

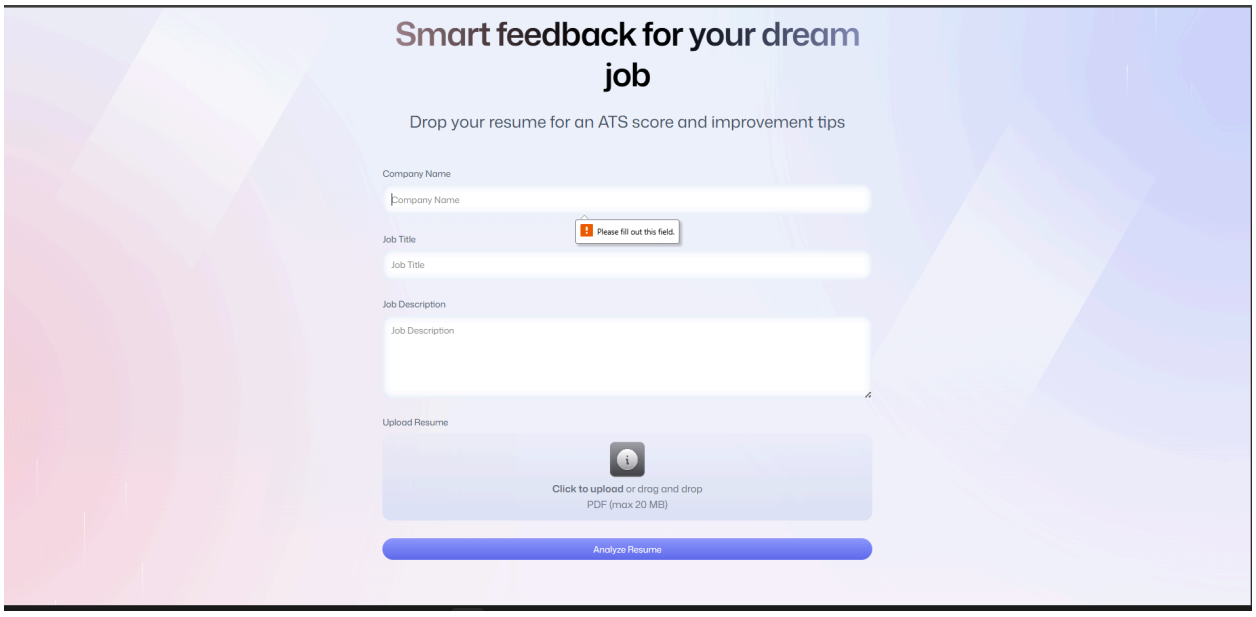

Figure: Error Handling & Alert Banner

8.6 GitHub Active Repository Matrix

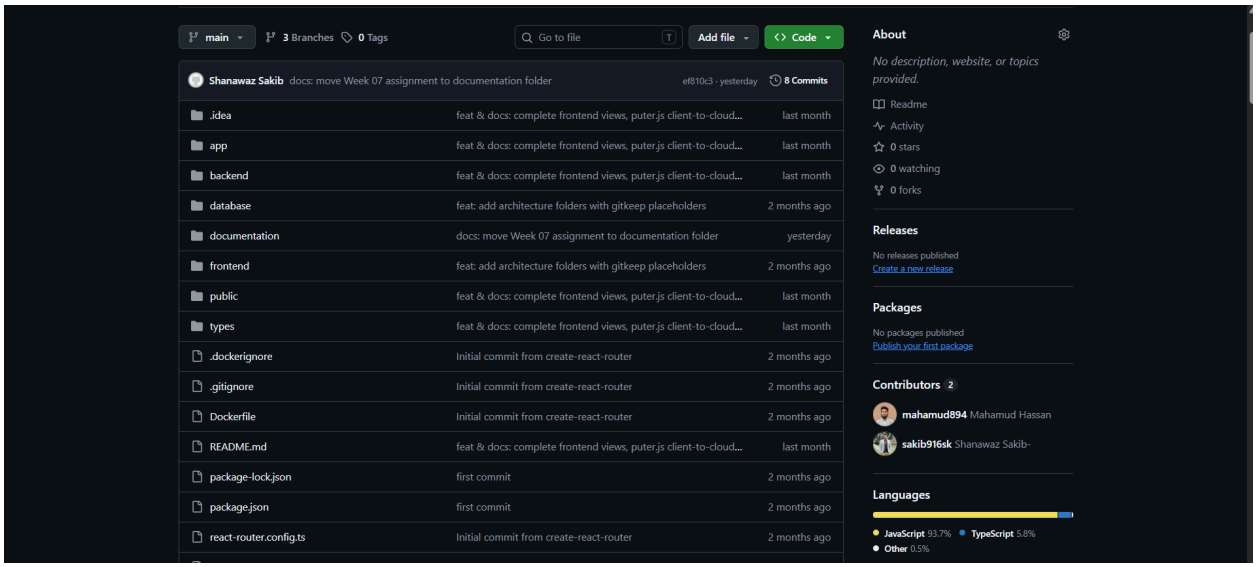

Figure: GitHub Repository